Two-Dimensional Privacy-Preserving Federated Learning Scheme Against Poisoning Attacks

Ke Gu, *Member, IEEE*, Wenwu Zhao, Jingjing Tan, *Member, IEEE*, Zheng Yang, Xiong Li, *Senior Member, IEEE*, and WeiJia Jia, *Fellow, IEEE*

***Abstract*—Federated learning is a distributed machine learning mechanism that allows multiple clients to collaboratively train a model with a central server. However, federated learning introduces new security challenges: 1) Some attackers can masquerade as the clients to poison the model training of federated learning, which poses a significant threat to the usability of the model; 2) A semi-honest server may infer and obtain clients' privacy data by launching inference attacks on their local gradients. To address these issues, we propose a two-dimensional privacy-preserving federated learning scheme against poisoning attacks, which can detect malicious gradients through two-dimensional detection procedure and protect the clients' privacy data by homomorphic encryption in the model training. In our two-dimensional detection procedure, the kernel density estimation and Euclidean distance are used to calculate the benign probability of each local gradient in the global detection stage. Subsequently, the hyper mesh detection structure is constructed in the local detection stage, where the benign probability of each local gradient is calculated by analyzing the relationship between each local gradient and the global gradient of the last round. Additionally, we construct an improved global model aggregation method to improve the convergence speed of the global model and ensure the prediction accuracy of the global model, which weights the corresponding gradient based on the reliability of each local gradient. Related experiments show that our scheme can effectively defend against poisoning attacks (such as Byzantine attacks and Label-flipping attacks).**

***Index Terms*—Federated learning, poisoning attack, privacy leakage, two-dimensional detection.**

I. Introduction

A. Background

With the development of science and technology, electronic devices have become integral to people's daily lives, generating massive amounts of data. By processing and analyzing the data, valuable information can be extracted to help people make their behavioral decisions. Furthermore, the emergence of machine learning has facilitated the processing of massive data, which makes machine learning widely used in daily life. Furthermore, since the data size owned by a single individual is relatively small, models trained solely on such data may suffer from poor generalization, federated learning has been extensively proposed and researched [1], [2]. As shown in Fig. 1, the clients only need to upload their local model parameters to the server for completing the global model training in the federated learning process, without sharing their local data. However, some research [3], [4] have shown that a semi-honest server can use the local gradients of clients and the global model to infer some clients' privacy information.

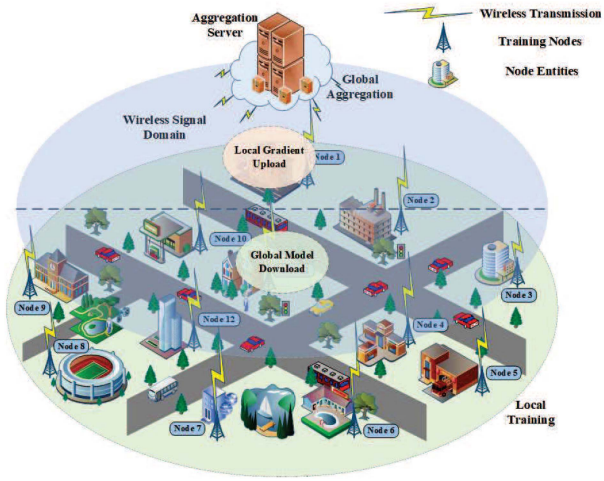


Fig. 1. Federated learning system framework.

Additionally, since federated learning requires multiple clients to participate in model training, it is vulnerable to some attackers (poisoners) whose goals are to influence the entire training process by uploading some malicious local gradients to the server for participating in the global aggregation. Therefore, a secure federated learning framework must ensure at least two key aspects: 1) The information privacy of the clients; 2) The security of the training process of the global model.

To ensure the security of clients' privacy information, many existing works [5], [6], [7], [8], [9], [10] mainly focus on how to process and obfuscate the true local gradients. These schemes are primarily based on three underlying technologies:

Received 4 August 2025; revised 24 November 2025; accepted 14 January 2026. Date of publication 21 January 2026; date of current version 12 May 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62572077, Grant U25A20429, and Grant 62332018, in part by the Open Research Fund of Hunan Provincial Key Laboratory of Intelligent Processing of Big Data on Transportation under Grant B202403, and in part byHunan Provincial Social Science Achievement Evaluation Committee under Grant XSP25YBC686. *(Corresponding authors: Jingjing Tan; Zheng Yang; Xiong Li.)*

Ke Gu, Wenwu Zhao, Jingjing Tan, and Zheng Yang are with the School of Computer Science and Technology, Changsha University of Science and Technology, Changsha 410114, China (e-mail: gk4572@163.com; 2642551601@qq.com; tanjingjing@csust.edu.cn; 2192158081@qq.com).

Xiong Li is with the Institute for Cyber Security, School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: lixiongzhq@163.com).

WeiJia Jia is with the Institute of Artificial Intelligence and Future Networks, Beijing Normal University, Zhuhai 519087, China (e-mail: jiawei_1950@hotmail.com).

Digital Object Identifier 10.1109/TDSC.2026.3656703

1545-5971 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

differential privacy [5], [6], secure multi-party computing [7], [8] and homomorphic encryption [9]. Schemes based on differential privacy typically work by adding noise to gradients, which can affect the performance of the trained models to a certain extent. The secure multi-party computing-based schemes require that the clients participating in the training interact with each other, which results in significant communication overhead. Homomorphic encryption-based schemes can protect client gradients with lower communication overhead, thus offering distinct advantages. Additionally, in poisoning attacks, the attackers mainly influence the global model's performance by uploading some malicious local gradients to the server. The attack methods are generally divided into targeted attacks (i.e. Label-flipping attacks) [11] and untargeted attacks (i.e. Byzantine attacks) [12]. In targeted attacks, the adversary aims to cause the trained global model to misclassify specific inputs into a predefined target class. In contrast, the goal of an untargeted attack is to degrade the overall prediction accuracy of the global model or to slow its convergence.

Currently, there is a large distributional differences between local models (or gradients) uploaded by targeted and untargeted attacks in federated learning. Therefore, most existing poisoning attack defense schemes are effective against only one type of attack, while few can defend against both [13], [14], [15]. Additionally, most of the defense schemes do not consider the issue of semi-honest servers. Semi-honest servers can obtain the clients' data through inference attacks on the local gradients. To address the above security issues, this paper constructs an effective privacy-preserving federated learning scheme that defends against poisoning attacks while simultaneously guaranteeing both the security of the model training process and the privacy of the clients.

B. Our Contributions

In this paper, we propose a two-dimensional privacy-preserving federated learning scheme against poisoning attacks, which can protect the clients' privacy data by homomorphic encryption and detect malicious gradients by two-dimensional detection method in the model training. In our scheme, the global detection stage uses kernel density estimation and Euclidean distance to calculate the benign probability of each local gradient. Subsequently, for the local detection stage, we construct a hyper-mesh detection structure where this probability is calculated by analyzing the relationship between each local gradient and the global gradient from the previous round. Furthermore, we construct an improved global model aggregation method to weight the corresponding gradient based on the reliability of each local gradient. Our main contributions are listed as follows:

- We propose a two-dimensional detection method for data poisoning attacks, which can detect local gradients accurately according to the distribution of local gradients and the update direction of each local gradient. In the two-dimensional detection process, the kernel density estimation and Euclidean distance are used to calculate the benign probability of each local gradient in the global detection stage; the hyper-mesh detection structure is constructed in the local detection stage, where the benign probability of each local gradient is calculated by analyzing the relationship between each local gradient and the global gradient of the last round.
- We construct an improved global model aggregation method, which weights the corresponding gradient based on the reliability of each local gradient. The improved aggregation method can adjust the update amplitude of the global gradient in each round by comparing the reliabilities of the local gradients participating between the current round and the last round, which can improve the convergence speed of the global model and ensure the prediction accuracy of the global model.
- We analyze the correctness and security of our scheme, and conduct extensive experiments on two real-world datasets to evaluate the performance of our scheme against poisoning attacks. Related experimental results show that our scheme is effective against two types of poisoning attacks (Byzantine attacks and Label-flipping attacks).

C. Organizations

The rest of this paper is organized as follows. In Section II, we review the related works about defense schemes against poisoning attacks. In Section III, we describe the employed methods. In Section IV, we show a model training framework under the setting of privacy-preserving federated learning, and define its security issues. In Section V, we provide a two-dimensional privacy-preserving federated learning scheme against poisoning attacks. In Section VI, we analyze the correctness and security of our scheme. We conduct some experiments to test the performance of our scheme in Section VII. Finally, we draw our conclusions in Section VIII.

II. Related Work

Currently, because the security of artificial intelligence is a research hotpots, many researchers proposed their privacy-preserving schemes and defense schemes against poisoning attacks in federated learning.

A. Privacy-Preserving Federated Learning

In order to hide the clients' privacy during the federated learning process, the existing solutions are mainly based on the following three basic technologies: differential privacy [6], secure multi-party computation [7] and homomorphic encryption [9]. The schemes that achieve privacy-preserving using differential privacy need to balance model performance and privacy-preserving. Liu et al. [5] sought the optimal balance between security and accuracy by adding adaptive noises. To solve the issue that attackers can obtain clients' privacy information by analyzing the parameters uploaded by the clients, Wei et al. [16] added artificial noise to the uploaded parameters before aggregation. Additionally, in combination with additive homomorphic encryption, Phong et al. [17] applied asynchronous stochastic gradient descent to neural networks to prevent privacy information from leaking to the server and ensure the accuracy

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

of the trained model. Miao et al. [18] leveraged Fully Homomorphic Encryption (FHE) to securely compute cosine similarity on ciphertexts for identifying malicious gradients. Chen et al. [19] designed a special packing scheme compatible with RLWE-based homomorphic encryption, which enables the efficient computation of key metrics required for robust aggregation. Liu et al. [20] employed homomorphic encryption technology and solved the contradictory relationship between protecting client privacy and defending against poisoning attacks.

B. Poisoning Attack Defenses in Federated Learning

In generally, poisoning attacks in federated learning mainly include target attacks and untarget attacks. Currently, many efforts have been made to identify malicious gradients in federated learning. Chen et al. [21] reduced the success and persistence of backdoor attacks by deleting the decision-making logic in the federated model. Wang et al. [22] performed four processing stages on the local model to defend against backdoor attacks. Hou et al. [23] established multiple filters and randomly sent them to the clients to prevent poisoning attacks. Zhu et al. [24] generated fake samples with backdoor features based on generative adversarial networks, and corrected the global model through knowledge distillation. Jebreel et al. [25] weighted the local gradients according to the relationship between the local gradients of the clients and the centroid of the compressed similarity vector. Li et al. [26] proposed the automatic weighted geometric mean method for Byzantine attacks, and designed an algorithm based on alternating optimization strategy. Cao et al. [27] grouped the clients and learned a model for each group of clients, and performed majority voting in the model to classify the test input.

Since most of the existing poison attack defense schemes can not defend against both target attacks and untarget attacks simultaneously, and the aggregation server may pose a threat to the privacy of clients. Therefore, we focus on constructing an effective privacy-preserving federated learning scheme against poisoning attacks, whose target is to provide the securities of the clients' privacy information and the training process.

III. Preliminaries

A. Federated Learning

In federated learning, multiple entities (clients) jointly and collaboratively train a global model with a central server [28]. A federated learning framework usually includes an aggregation server and N clients $U = \{u_1, u_2, \ldots, u_N\}$, where $\mathcal{N} = \{1, 2, \ldots, N\}$. The training procedure of the federated model generally includes the following steps:

- *Step 1:* The aggregation server initializes the system before training model, sets the network model $\boldsymbol{w}^0$ used by each client for the model training, and selects the qualified clients to participate in the model training.
- *Step 2:* Each client u_i downloads the initial model $\boldsymbol{w}^{t-1}$ of each training round from the aggregation server and trains its own local gradient $\boldsymbol{l}_i^t$ through its local dataset. Then the client u_i uploads the local gradient $\boldsymbol{l}_i^t$ of this round to the aggregation server.
- *Step 3:* After receiving the local gradient $\boldsymbol{l}_i^t$ uploaded by client u_i for $i \in \mathcal{N}$, the aggregation server aggregates $\{\boldsymbol{l}_1^t, \boldsymbol{l}_2^t, \ldots, \boldsymbol{l}_N^t\}$ to obtain a new global model $\boldsymbol{w}^t$.
- *Step 4:* Repeat **Step2** to **Step3** until the training requirements are met.

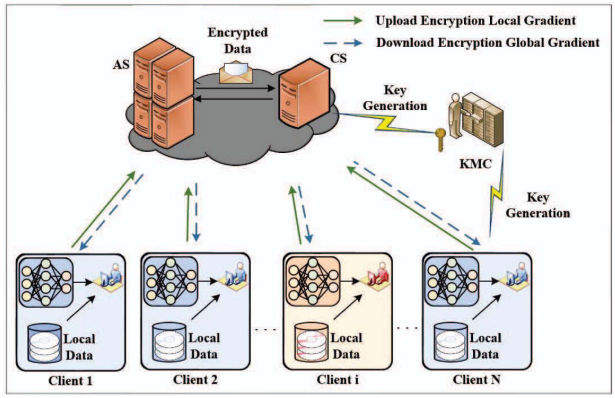


Fig. 2. System framework of privacy-preserving federated learning.

B. Linear Homomorphic Encryption

Homomorphic encryption (HE) refers to performing specific calculation operations on obtained ciphertexts after homomorphic encryption of original data, and homomorphic decryption refers to decrypt the calculated ciphertexts to correctly obtain the plaintext results equivalent to the results by calculating the original plaintext data [29]. Linear homomorphic encryption includes four algorithms: 1) $KeyPro$: a algorithm to generate key pairs; 2) Enc: an encryption algorithm; 3) Dec: a decryption algorithm; 4) Cal: a calculation algorithm for ciphertexts in a homomorphic encryption process. Further, the linear homomorphic encryption satisfies the standard security definition (semantic security). Namely, the ciphertext does not reveal any information in the plaintext. For example, in the Paillier homomorphic encryption process [29], given two plaintexts x_1 and x_2, a public key pk and a constant p, we can have the following security assurance: (a) $[[x_1]]_{pk} \cdot [[x_2]]_{pk} = [[x_1 + x_2]]_{pk}$; (b) $[[x_1]]_{pk}^p = [[p \cdot x_1]]_{pk}$. As seen above, we use $[[\cdot]]_{pk}$ to represent the data encrypted by pk in this paper.

IV. System Framework and Problem Formulation

In this section, we show a model training framework under the setting of privacy-preserving federated learning, and define its security issues (including poisoning attacks).

A. System Framework

In this section, we construct a privacy-preserving federated learning system framework. As we can see in Fig. 2, in this system framework, there are four entities: aggregation server, collaboration server, key management center and client. whose functions are described as follows:

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

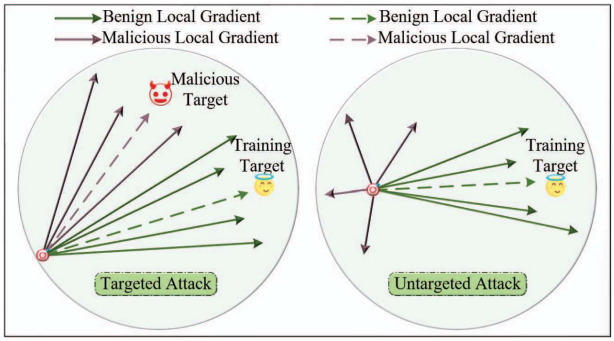


Fig. 3. Attack targets.

- *Aggregation server (AS):* The aggregation server has three main functions: 1) it can set up the initial network model and select eligible clients for global model training; 2) it can collect the encrypted local gradients uploaded by each client, and can cooperate with the collaboration server to detect the encrypted malicious gradients under the homomorphic encryption environment; 3) it can aggregate the benign encrypted local gradients to get a credible global gradient for updating the global model.
- *Collaboration server (CS):* To ensure the privacy of local gradients, the collaboration server assists the aggregation server in completing the relevant calculations on the encrypted data.
- *Key management center (KMC):* The key management center is a trusted authority, whose functions mainly includes generating, distributing and managing public-private key pairs for related entities in this system.
- *Client:* Each client owns its local dataset. In each training round, the client can receive the initial model sent by the aggregation server and perform local model training with its local dataset. Further, each client encrypts and sends its local gradients to the aggregation server. What's more, the ciphertext of the global model parameters can be decrypted by eligible clients.

B. Threat Model

In this section, we analyze and show the security issues for our model training framework under the setting of privacy-preserving federated learning, which include how to defend poisoning attacks and provide the function of privacy-preserving in the model training process. Based on previous work [30], [31], [32], [33], [34], we assume that the aggregation server and the collaboration server are honest-but-curious and non-colluding. This is a common assumption in real-world application scenarios, as the two servers can be different but mutually constrained entities. In our model training framework, we consider a threat model where a minority of clients may be fully malicious and capable of colluding to launch coordinated attacks. As illustrated in Fig. 3, these attacks are broadly classified into two types: targeted and untargeted. In a targeted attack, multiple adversarial clients collaborate to manipulate the global model's behavior toward a specific objective, such as causing misclassification on particular inputs. In contrast, an untargeted attack aims to degrade the overall performance and convergence of the global model, often without requiring explicit collusion among malicious clients. The formal definitions of these attack objectives are provided in Definition 1.

Definition 1. (Attack Targets): For the targets of untarget attacks and target attacks, the targets of all attackers can uniformly be expressed as:

$$y(x) = \begin{cases} \widetilde{y}(x), Malicious\ target \\ \widehat{y}(x), Benign\ behaviour \end{cases} \tag{1}$$

where $y(x)$ is the predicted class of sample x by global model, $\widehat{y}(x)$ is the real label value corresponding to the sample x; the attackers' goals are to control the global model to predict the class of the sample x as $\widetilde{y}(x)$, with $\widehat{y}(x) \neq \widetilde{y}(x)$. Further, each attacker's $\widetilde{y}(x)$ in untarget attacks is different, and $\widetilde{y}(x)$ is the same for all attackers in target attacks.

Similar to the works [35], [36], [37], we assume that the proportion of poisoners does not exceed a limited threshold. Since aggregation server can receive local gradients from all clients participating in the federated learning training, we consider that it is semi-trusted (honest and curious), namely it can perform all calculation processes honestly, but it is interested in the clients' information contained in each local gradient. Therefore, we set up an additional collaboration server to assist the aggregation server with some computations in an encrypted environment. Like the aggregation server, it is also semi-trusted, and it does not collude with the aggregation server [9]. Our aim in this paper is to ensure the global model's prediction accuracy and the security of the clients' privacy information [38], [39], [40].

V. Two-Dimensional Privacy-Preserving Federated Learning Scheme

In this section, we propose a two-dimensional privacy-preserving federated learning scheme against poisoning attacks, which specifically includes four steps: system initialization, local training, two-dimensional poisoning attack detection and global model update. To detect malicious local gradients uploaded by attackers and protect the privacy information of all clients, our proposed scheme can detect malicious local gradients against poisoning attacks from two dimensions: global detection (Dimension-1) and local detection (Dimension-2). In global detection stage, kernel density estimation (KDE) and Euclidean distance are used to calculate the benign probability of each local gradient. In local detection stage, the benign probability of each local gradient is calculated by analyzing the relationship between each local gradient and the global gradient of the last round. Based on the detection results under the global and local detections, our scheme can screen out malicious local gradients and assign the weights to the remaining local gradients. Moreover, in order to further improve the global model's convergence speed, we design a global model update method. Additionally, we construct a homomorphic encryption method to process the calculation procedures in an encrypted environment, which can further protect the privacy information of the clients.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SYMBOL AND DESCRIPTION

Symbols	*Descriptions*
k	*security parameter*
pk	*public key*
sk	*private key*
$\boldsymbol{l}$	*local gradient*
$\boldsymbol{g}$	*global gradient*
$\boldsymbol{w}$	*global model*
α	*benign probability obtained from global detection*
β	*benign probability obtained from local detection*
γ	*reliability of local gradient*

The commonly used symbols and their descriptions in this paper are shown in Table I.

A. System Initialization

In this section, we detail the initialization procedure of system. For each registered client u_i who can participate in model training, the key management center generates a pair of asymmetric keys (pk_i, sk_i) based on the linear homomorphic encryption [29]. Also, the key management center generates a pair of asymmetric keys (pk_c, sk_c) for the collaboration server based on the linear homomorphic encryption. Further, the aggregation server sets a initial global model $\boldsymbol{w}^0$, then uses each client's public key pk_i to encrypt and send the initial global model $\boldsymbol{w}^0$ to each client.

B. Local Training

In this stage, each client u_i that participates in model training decrypts the received initial model with its private key sk_i, and trains a local model with its local dataset to obtain a local gradient, where the local gradient is encrypted to the aggregation server for training a new global model. For any t-th training round, the detailed process is described as follows:

- *Step 1:* Each client u_i decrypts the encrypted $(t-1)$-th global model $[[\boldsymbol{w}^{t-1}]]_{pk_i}$ from the aggregation server by its private key sk_i, and trains the local model with its local dataset to obtain a local gradient $\boldsymbol{l}_i^t$;
- *Step 2:* Each client u_i uses the public key pk_c of the collaboration server to encrypt the local gradient $\boldsymbol{l}_i^t$ to obtain $[[\boldsymbol{l}_i^t]]_{pk_c}$. Then each client u_i sends the ciphertext $[[\boldsymbol{l}_i^t]]_{pk_c}$ to the aggregation server. According to the work [9], we assume each obtained gradient value $\boldsymbol{l}_i^t$ is an integer in this paper[1].

C. Two-Dimensional Poisoning Attack Detection

In this section, we propose a two-dimensional poisoning attack detection scheme, which is to prevent malicious local gradients from adversely influencing the global model training. In our scheme, the aggregation server receives the ciphertext $[[\boldsymbol{l}_i^t]]_{pk_c}$ of the uploaded gradient from each client at the t-th round of model training, and then can obtain an encrypted local gradient dataset $\mathcal{L}^t = \{[[\boldsymbol{l}_1^t]]_{pk_c}, [[\boldsymbol{l}_2^t]]_{pk_c}, \ldots, [[\boldsymbol{l}_N^t]]_{pk_c}\}$. Then the aggregation server performs the global detection (Dimension-1) and the local detection (Dimension-2) with the assistance of the collaboration server. Through the two round of detections, two benign probabilities α_i^t and β_i^t for each local gradient $\boldsymbol{l}_i^t$ can be obtained,[2] respectively. If α_i^t and β_i^t are both smaller than the corresponding first-quartile of all local gradients, $\boldsymbol{l}_i^t$ is considered to be malicious. By this method, we can obtain a malicious local gradient set $\mathcal{L}^{\mathcal{M}}$. Further, based on the calculation of α_i^t and β_i^t, the reliability γ_i^t of each remaining local gradient that does not belong to $\mathcal{L}^{\mathcal{M}}$ is calculated, which is used to weight the corresponding local gradient in the global model aggregation, and the weights of the local gradients in $\mathcal{L}^{\mathcal{M}}$ are set to 0.[3]

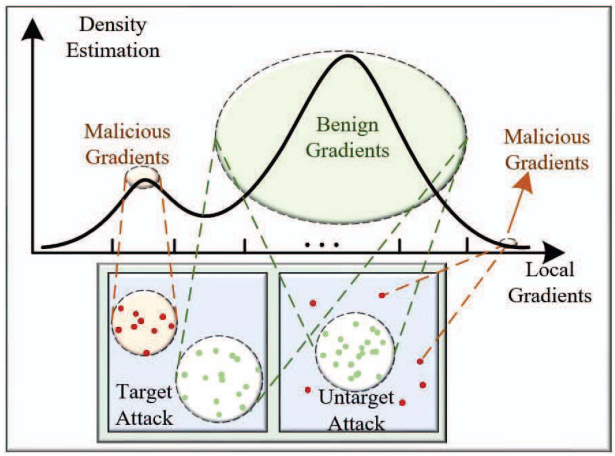


Fig. 4. Principle of global detection.

1) Global Detection (Dimension-1): In this section, we show a detailed global detection procedure. Since the local data owned by each client are independent and identically distributed, the training goal and model structure of each client are consistent, thus each obtained local gradient is also independent and identically distributed. However, the distribution of malicious local gradients is different from that of the normal gradients. So, although the real distribution of all local gradients is unknown, the density estimation of each local gradient can be used as a parameter to measure whether it is malicious, as shown in Fig. 4.

To prevent the disclosure of clients' private data, we design a global detection protocol, as outlined in Algorithm 1. This protocol is jointly executed by the aggregation server and the collaboration server to identify potential threats. The detailed steps are described as follows:

- The aggregation server randomly selects a non-zero integer r, and then encrypts r with the collaboration server's public key pk_c. Then the aggregation server obtains the mask value $\boldsymbol{L}_i^t$ of $[[\boldsymbol{l}_i^t]]_{pk_c}$ by calculating $[[\boldsymbol{l}_i^t]]_{pk_c} \cdot [[r]]_{pk_c}$.

[1] That is because our employed Paillier password system works in the integer domain.

[2] In our scheme, in order to protect the privacy information of all clients, all these computing processes are jointly completed by the aggregation server and the collaboration server in encrypted environment.

[3] In this work, since we first assume only a portion of malicious gradients is filtered out in the detection procedure, there still may be few malicious gradients in the remaining local gradients. So, the remaining local gradients are weighted to aggregate so as to further eliminate the influence of malicious gradients.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Global Detection.

Input : Gradient $\{[[\boldsymbol{l}_i^t]]_{pk_c}\}_{i=1}^{i=N}$ owned by AS; Private key sk_c owned by CS.
Output: Benign probability α_i^t of $[[\boldsymbol{l}_i^t]]_{pk_c}$.

Procedure:
  Aggregation server (AS):
  $r \leftarrow$ Select a non-zero integer randomly
  for $i \in \mathcal{N}$ **do**
    $\boldsymbol{L}_i^t = [[r]]_{pk_c} \cdot [[\boldsymbol{l}_i^t]]_{pk_c}$
    Send $\boldsymbol{L}_i^t$ to CS
  end for
  Collaboration server (CS):
  for $i \in \mathcal{N}$ **do**
    $\mathbb{L}_i^t = Dec(sk_c, \boldsymbol{L}_i^t)$
  end for
  $m = 1 \leftarrow$ Initialize the index of the baseline
  for $i \in \mathcal{N}$ **do**
    Calculate q_i^t of $\mathbb{L}_i^t$ according to Eq. (2)
    if $q_i^t > q_m^t$ **then**
      $m = i$
    end if
  end for
  for $i \in \mathcal{N}$ **do**
    $\mathcal{D}_i^t = \text{dist}(\mathbb{L}_i^t, \mathbb{L}_m^t)$
  end for
  for $i \in \mathcal{N}$ **do**
    $\alpha_i^t = \frac{\mathcal{D}_b^t}{\mathcal{D}_i^t}, \mathcal{D}_b^t = \min_{i \in [1,N]}(\mathcal{D}_i^t)$
  end for

- The aggregation server sends $\boldsymbol{L}_i^t$ with $i \in \mathcal{N}$ to the collaboration server. After the collaboration server receives $\boldsymbol{L}_i^t$, the collaboration server uses its private key sk_c to decrypt $\boldsymbol{L}_i^t$ to obtain $\mathbb{L}_i^t$.

Then, we calculate the kernel density estimate of each local gradient to obtain the probability density q_i^t of each local gradient distributed of all local gradients as follows:

$$q_i^t = \frac{1}{N \cdot h} \cdot \sum_{j=1}^{N} K(\mathbb{L}_i^t, \mathbb{L}_j^t) \tag{2}$$

where $K(\cdot)$ is a kernel function, and h is a smoothing parameter with $h > 0$ (also known as the bandwidth of the function $K(\cdot)$).

In general, when different kernel functions are used, the corresponding bandwidths are different. In a Gaussian function, the points that are further away from the centre have smaller weight and the points closer to the centre have larger weight. And the local gradients are independent and identically distributed, which can be approximated as normal distribution. Therefore, we use the Gaussian function as kernel function. Further, Silverman [41] concludes that the optimal bandwidth of Gaussian kernel function is $1.05 \cdot \hbar \cdot N^{-1/5}$, where $\hbar$ is the standard parameter deviation of the same dimension in all the local gradients. Therefore, we set $h = 1.05 \cdot \hbar \cdot N^{-1/5}$. The expression of the Gaussian kernel function is as follows:

$$K(\mathbb{L}_i^t, \mathbb{L}_j^t) = \frac{1}{\sqrt{2 \cdot \pi}} \cdot e^{\frac{-(\mathbb{L}_i^t - \mathbb{L}_j^t)^2}{2 \cdot h^2}} \tag{3}$$

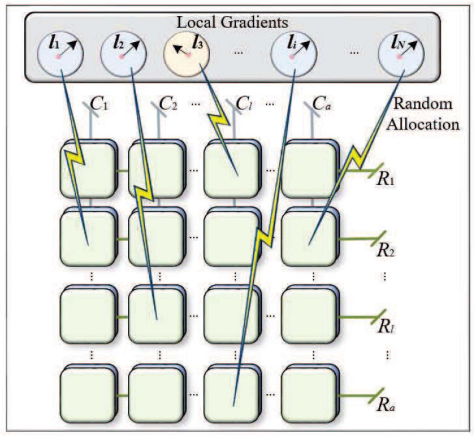


Fig. 5. Two dimensional hyper mesh structure.

According to the property of kernel density estimation, we can learn that the density estimation is the larger, the more data points correspond to the same region. Therefore, the density of the region where each local gradient is located can be determined according to the density estimation. Further, we use the local gradient whose density estimation is the maximal value as the baseline and denote it as $\mathbb{L}_m^t$. Then we calculate the Euclidean distance between each local gradient $\mathbb{L}_i^t$ and $\mathbb{L}_m^t$ as follows (i and m are the indexes of different clients respectively):

$$\mathcal{D}_i^t = \text{dist}(\mathbb{L}_j^t, \mathbb{L}_m^t) \tag{4}$$

where $\text{dist}(\cdot)$ is the function to calculate the Euclidean distance. When the Euclidean distance is larger, the corresponding local gradient has a higher probability of being malicious. Finally, the collaboration server calculates the benign probability[4] α_i^t of each local gradient based on $\mathcal{D}_i^t$:

$$\alpha_i^t = \frac{\mathcal{D}_b^t}{\mathcal{D}_i^t} \tag{5}$$

where $\mathcal{D}_b^t = \min\{\mathcal{D}_1^t, \mathcal{D}_2^t, \ldots, \mathcal{D}_N^t\}$.

2) Local Detection (Dimension-2): In this section, we show a detailed local detection procedure. In the current federated learning, although the local gradients are different from each other, the benign local gradients are all updated towards the same training goal. Therefore, compared to the last round of global gradient, the angular change in the direction of each benign client's local gradient is similar. However, to complete the poisoning attack, the angle change of the malicious local gradients is different from that of the benign local gradients. In the local detection, the aggregation server first groups the local gradients according to the two dimensional hyper mesh (as shown in Fig. 5), and then the local malicious detection is performed on each gradient group. The main local detection procedure is shown in Fig. 6.

Therefore, we construct a local detection protocol (as shown in Algorithm 2) with protecting the privacy of client, which includes two stages: the first one is to group the local gradients; the second one is to calculate the benign probability β_i^t of each

[4] we set the α_i^t of $\boldsymbol{l}_m^t$ as 1.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

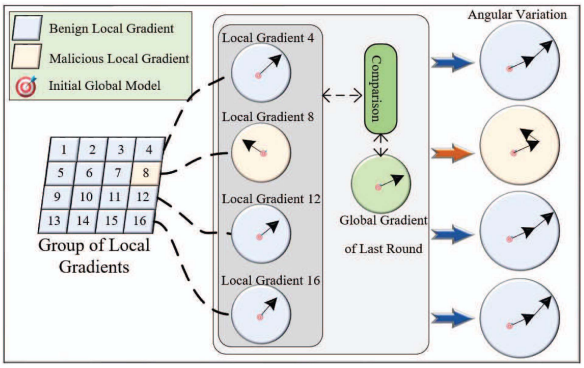


Fig. 6. Principle of local detection.

local gradient in its group, which is further used for malicious gradient detection. The two stages are described in detail as follows.

a) Grouping of Local Gradients: The aggregation server firstly calculates $[[\boldsymbol{l}_i^t]]_{pk_c}^{r_g}$ to obtain the corresponding mask value $\boldsymbol{L}_i^t = [[r_g \cdot \boldsymbol{l}_i^t]]_{pk_c}$, where r_g is a random non-zero integer. Then the aggregation server sends $\boldsymbol{L}_i^t$ to the collaboration server. The collaboration server decrypts $\boldsymbol{L}_i^t$ by its private key to obtain $\mathbb{L}_i^t$, and then groups $\mathbb{L}_i^t$ for all is with $i \in \mathcal{N}$. Further, the detailed grouping process is described as follows: the collaboration server builds a two dimensional hyper mesh (as shown in Fig. 5) according to the following Formulas (6) and (7), where each unit in the mesh belongs to two groups of rows and columns simultaneously.

$$\mathcal{R}_h = \begin{cases} \lfloor N^{\frac{1}{2}} \rfloor, 0 < h < \lfloor N^{\frac{1}{2}} \rfloor \ \& \ N < \lfloor N^{\frac{1}{2}} \rfloor \cdot \lceil N^{\frac{1}{2}} \rceil \\ \lceil N^{\frac{1}{2}} \rceil, 0 < h < \lfloor N^{\frac{1}{2}} \rfloor \ \& \ N > \lfloor N^{\frac{1}{2}} \rfloor \cdot \lceil N^{\frac{1}{2}} \rceil \\ N - \lfloor N^{\frac{1}{2}} \rfloor^2, h = \lfloor N^{\frac{1}{2}} \rfloor + 1 \ \& \ N < \lfloor N^{\frac{1}{2}} \rfloor \cdot \lceil N^{\frac{1}{2}} \rceil \\ N - \lfloor N^{\frac{1}{2}} \rfloor \cdot \lceil N^{\frac{1}{2}} \rceil, h = \lfloor N^{\frac{1}{2}} \rfloor + 1 \ \& \ N > \lfloor N^{\frac{1}{2}} \rfloor \cdot \lceil N^{\frac{1}{2}} \rceil \\ N^{\frac{1}{2}}, Others \end{cases} \tag{6}$$

$$\mathcal{C}_l = \begin{cases} \lceil N^{\frac{1}{2}} \rceil, 0 < l < N - \lfloor N^{\frac{1}{2}} \rfloor^2 \ \& \ N < \lfloor N^{\frac{1}{2}} \rfloor \cdot \lceil N^{\frac{1}{2}} \rceil \\ \lceil N^{\frac{1}{2}} \rceil, 0 < l < N - \lfloor N^{\frac{1}{2}} \rfloor \cdot \lceil N^{\frac{1}{2}} \rceil \ \& \ N > \lfloor N^{\frac{1}{2}} \rfloor \cdot \lceil N^{\frac{1}{2}} \rceil \\ \lfloor N^{\frac{1}{2}} \rfloor, Others \end{cases} \tag{7}$$

where h and l represent the index of row and column respectively; $\mathcal{R}_h$ and $\mathcal{C}_l$ represent the number of units in the h-th row and the l-th column respectively. Therefore, take Fig. 5 as an example, we use R_1 to the group of the 1-th row, which includes the number $\mathcal{R}_1$ of units; we use C_2 to the group of the 2-th column, which includes the number $\mathcal{C}_2$ of units, respectively.

Algorithm 2: Local Detection.

Input : Gradient $\left[\left[\boldsymbol{l}_i^t\right]\right]_{pk_c}, i \in \mathcal{N}$ owned by AS; Parameters sk_c, $\mathbb{G}^{t-1}$ (mask of $\boldsymbol{g}^{t-1}$ calculated in Algorithm 4) owned by CS.
Output: Benign probability β_i^t of $\left[\left[\boldsymbol{l}_i^t\right]\right]_{pk_c}, i \in \mathcal{N}$.

Procedure:
Stage 1: Grouping of Local Gradients
Aggregation server (AS):
for $i \in \mathcal{N}$ **do**
  $r_g \leftarrow$ Select a non-zero integer randomly
  $\boldsymbol{L}_i^t = [[\boldsymbol{l}_i^t]]_{pk_c}^{r_g}$
  Send $\boldsymbol{L}_i^t$ to CS
end for
Collaboration server (CS):
Build d^2-hyper mesh according to N
for $i \in \mathcal{N}$ **do**
  $\mathbb{L}_i^t = Dec(sk_c, \boldsymbol{L}_i^t)$
  Randomly assign a position for $\mathbb{L}_i^t$
end for
Stage 2: Calculation of Benign Probability β_i^t
Collaboration server (CS):
for $i \in \mathcal{N}$ **do**
  /* ▷Calculate the Cosine similarity */
  $\theta_i^t = \frac{\mathbb{L}_i^t \cdot \mathbb{G}^{t-1}}{||\mathbb{L}_i^t|| \cdot ||\mathbb{G}^{t-1}||}$
end for
Calculate $\mathcal{X}_i^t$ and $\mathcal{Y}_i^t$ of $\mathbb{L}_i^t$ according to Eq. (9) and Eq. (10)
for $i \in \mathcal{N}$ **do**
  Calculate β_i^t of $\mathbb{L}_i^t$ according to Eq. (11)
end for

Further, according to the constructed two dimensional hyper mesh, the encrypted local gradient $[[\boldsymbol{l}_i^t]]_{pk_c}$ uploaded by each client is randomly putted into one unit in the mesh. We use (h_i, l_i) to represent the position of $[[\boldsymbol{l}_i^t]]_{pk_c}$. In addition, the attackers may reason to find the best suitable attacked clients so as to be easier to escape the related detections (after implementing poisoning attacks), thus the positions of all the local gradients are changed in each round of local detection to increase the randomness of local detection in our scheme.

b) Calculation of Benign Probability β_i^t: In the federated learning, the training goal of each client with the initial setting of environment and model parameters is the same, and the training data is independent and identically distributed. Therefore, in each training round, all the benign clients can generate their local gradients with similar angular changes in the update direction compared to the last round of the global gradient, as shown in Fig. 6. Then we calculate the benign probability β_i^t of each local gradient by comparing the angle between the local gradient generated by each client and the global gradient of the last round, which is further used for malicious gradient detection. In this work, due to the angle and cosine similarity have similar meanings, they both can reflect the angle relationship between two vectors. Therefore, for the convenience of calculation, we replace the angle with the cosine similarity. The specific process is described as follows:

- The cosine similarity θ_i^t between $\boldsymbol{l}_i^t$ and $\boldsymbol{g}^{t-1}$ is calculated by the collaboration server as follows:

$$\theta_i^t = \frac{\boldsymbol{l}_i^t \cdot \boldsymbol{g}^{t-1}}{||\boldsymbol{l}_i^t|| \cdot ||\boldsymbol{g}^{t-1}||} \tag{8}$$

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

where $\boldsymbol{g}^{t-1}$ is the global gradient of $t-1$-th training round.

- According to θ_i^t, the collaboration server calculates the benign probability $\mathcal{X}_i^t$, $\mathcal{Y}_i^t$ of $\boldsymbol{l}_i^t$ in its row and column respectively:

$$\mathcal{X}_i^t = \frac{\vartheta_{i,h}^t}{\sum_{j \in R_{h_i}} \vartheta_{j,h}^t} \tag{9}$$

$$\mathcal{Y}_i^t = \frac{\vartheta_{i,l}^t}{\sum_{j \in C_{l_i}} \vartheta_{j,l}^t} \tag{10}$$

$$\text{s.t. } \vartheta_{i,h}^t = \exp\left(-k \cdot \left|\theta_i^t - \frac{1}{|\mathcal{R}_i|} \cdot \sum_{j \in \mathcal{R}_{h_i}} \theta_j^t\right|\right),$$

$$\vartheta_{i,l}^t = \exp\left(-k \cdot \left|\theta_i^t - \frac{1}{|\mathcal{C}_i|} \cdot \sum_{j \in \mathcal{C}_{l_i}} \theta_j^t\right|\right)$$

According to (6), we know that the range of α_i^t is [0,1], so the collaboration server calculates the confidence β_i^t based on $\mathcal{X}_i^t$ and $\mathcal{Y}_i^t$ and adjusts the range of β_i^t to [0,1]:

$$\beta_i^t = \min(\widehat{\mathcal{X}}_i^t, \widehat{\mathcal{Y}}_i^t) \tag{11}$$

where $\widehat{\mathcal{X}}_i^t = \frac{\mathcal{X}_i^t}{\max_{j \in \mathcal{N}}(\mathcal{X}_j^t)}$, $\widehat{\mathcal{Y}}_i^t = \frac{\mathcal{Y}_i^t}{\max_{j \in \mathcal{N}}(\mathcal{Y}_j^t)}$. The smaller value between $\widehat{\mathcal{X}}_i^t$ and $\widehat{\mathcal{Y}}_i^t$ is chosen because it can prevent malicious gradients from evading local detection. And we prove the effectiveness of our detection scheme in Theorem 1.

Theorem 1: Based on the two dimensional hyper mesh, our local detection method can defend against target attacks and untarget attacks.

Proof: Our local detection method can detect target attacks and untarget attacks according to the constructed two dimensional hyper mesh as follows:

- *Detection against target attacks:* If the (benign) probabilities of the malicious local gradients are bigger than those of the benign local gradients, the group that the malicious local gradients belong to must meet the following condition that the number of the malicious local gradients in the row and column is greater than the number of the benign local gradients. Then we assume as follows:
  - ⋆ $N = a^2$, where N is the number of local gradients and a is the size of the two dimensional hyper mesh.
  - ⋆ The number of the malicious local gradients is $M = \eta \cdot N$, where η is the proportion of poisoners (malicious local gradients) and $\eta < 0.5$.

  If a malicious local gradient can evade our local detection method, then in its row and column groups, the numbers of malicious local gradients need to be greater than or equal the numbers of the benign local gradients. Further, it can be known that when the numbers of the malicious local gradients equal the numbers of the benign local gradients in both row and column groups, the malicious local gradient can evade our local detection method with the maximal probability. Therefore, based on the constructed two dimensional hyper mesh, we can calculate the maximal probability that the malicious local gradient can evade as follows:

$$\begin{aligned} P &= \frac{\prod_{i=0}^{a}(M-i)}{\prod_{i=0}^{a/2}(N-a/2-i) \cdot \prod_{j=0}^{a/2}(N-3a/2-j)} \\ &= \frac{\prod_{i=0}^{a}(\eta a^2 - i)}{\prod_{i=0}^{a/2}(a^2-a/2-i) \cdot \prod_{j=0}^{a/2}(a^2-3a/2-j)} \\ &= \frac{\prod_{i=0}^{a}(\eta a - i/a)}{\prod_{i=0}^{a/2}(a-i/a-1/2) \cdot \prod_{j=0}^{a/2}(a-j/a-3/2)} \\ &< \frac{(\eta a)^2}{(a-2)^2} = \left(\frac{\eta a}{a-2}\right)^2 \end{aligned}$$

  From the above formula, we can know that the maximal probability is not more than $(\frac{\eta \cdot a}{a-2})^2$. When the number N of local gradients approaches to infinity, the formula $(\frac{\eta \cdot a}{a-2})^2$ approaches to 0. Thus, when N approaches to infinity, the probability that the malicious local gradient evades is close to 0.
- *Detection against untarget attacks:* In untarget attacks, the goal of each attacker is to send malicious local gradients to affect the model's convergence speed, and there is no relationship between malicious local gradients. Therefore, the change direction of each malicious local gradient in each group (compared with the global gradient of the last round) is different from that of other local gradients in the same group. Then our local detection method is easier to detect malicious gradients generated by untarget attacks according to the constructed two dimensional hyper mesh.

3) Process of Malicious Gradients: In this section, we show how to further screen out the malicious local gradients. In our scheme, after the aggregation server performs the global detection and the local detection with the assistance of the collaboration server, two benign probabilities α_i^t and β_i^t for each local gradient $\boldsymbol{l}_i^t$ can be obtained, respectively. If α_i^t and β_i^t are both smaller than the corresponding first-quartile of all local gradients, then the local gradient $\boldsymbol{l}_i^t$ is considered to be malicious. So, we can obtain a malicious local gradient set $\mathcal{L}^{\mathcal{M}}$, where these local gradients in $\mathcal{L}^{\mathcal{M}}$ can not participate in the global model aggregation. Since this detection of malicious gradients is only a preliminary one so that there may still be malicious local gradients to make the influence on global model training. Therefore, we need to further process the two benign probabilities α_i^t and β_i^t for each local gradient $\boldsymbol{l}_i^t$. After the global and local benign probabilities α_i^t and β_i^t for each local gradient is obtained, it can know which local gradient is malicious. Then, based on α_i^t and β_i^t, the reliability γ_i^t of each remaining local gradient that does not belong to $\mathcal{L}^{\mathcal{M}}$ is calculated, which

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: Local Gradient Process.

Input : Benign probability sets $\{\alpha_i^t\}_{i=1}^{i=N}$,$\{\beta_i^t\}_{i=1}^{i=N}$ owned by CS, $[[\boldsymbol{l}_i^t]]_{pk_c}$ owned by AS.
Output: A benign local gradient index-set $\mathcal{B}$ and a benign probability set $\mathcal{P}^t$ of benign local gradient.

Procedure:
  Collaboration server (CS):
  $\mathcal{S}_g$, $\mathcal{S}_l \leftarrow$ Sort $\{\alpha_i^t\}_{i=1}^{i=N}$,$\{\beta_i^t\}_{i=1}^{i=N}$ respectively
  $\mathcal{S}_g^*$, $\mathcal{S}_l^* \leftarrow$ Locate out the elements in $\mathcal{S}_g$ and $\mathcal{S}_l$ that are less than the first-quartile, respectively
  $\mathcal{M}_g^*$, $\mathcal{M}_l^* \leftarrow$ Local gradients corresponding to the saved indexes in $\mathcal{S}_g^*$ and $\mathcal{S}_l^*$, respectively
  $\mathcal{M} = \mathcal{M}_g^* \cap \mathcal{M}_l^* \leftarrow$ Index set of malicious gradients
  $\mathcal{B} = \mathcal{N} - \mathcal{M} \leftarrow$ Index set of benign gradients
  for $i \in \mathcal{B}$ **do**
    /* ▷Calculate reliability */
    $\gamma_i^t = \frac{1}{2} \cdot (\alpha_i^t + \beta_i^t)$
    Send γ_i^t to AS
  end for
  Aggregation server (AS):
  /* ▷The reliability set of benign local gradient */
  $\mathcal{P}^t = \{\gamma_i^t, i \in \mathcal{B}\}$

is further used to assign the weight to each local gradient to aggregate the global model. The specific process is below:

- The collaboration server sorts these elements in $\{\alpha_i^t\}$ and $\{\beta_i^t\}$ with $i \in \mathcal{N}$ from smallest to largest to obtain two sets $\mathcal{S}_{\mathrm{g}}$ and $\mathcal{S}_l$.
- The collaboration server locates out the elements that are less than the first-quartile of the two sets $\mathcal{S}_{\mathrm{g}}$ and $\mathcal{S}_l$ to obtain two subsets $\mathcal{S}_{\mathrm{g}}^*$ and $\mathcal{S}_l^*$. Based on $\mathcal{S}_{\mathrm{g}}^*$ and $\mathcal{S}_l^*$, the collaboration server can obtain two index-set $\mathcal{M}_{\mathrm{g}}^*$ and $\mathcal{M}_l^*$ corresponding to the benign probability of each element in $\mathcal{S}_{\mathrm{g}}^*$ and $\mathcal{S}_l^*$ respectively, where the local gradients in the two sets $\mathcal{M}_{\mathrm{g}}^*$ and $\mathcal{M}_l^*$ are considered to be malicious. By this method, the collaboration server can get an index-set $\mathcal{M}$ of malicious local gradients and an index-set $\mathcal{B}$ of benign local gradients, where $\mathcal{M} = \mathcal{M}_{\mathrm{g}}^* \cap \mathcal{M}_l^*$ and $\mathcal{B} = \mathcal{N} - \mathcal{M}$. And the local gradients (whose index is in $\mathcal{M}$) don't use to aggregate the global model.
- The reliability γ_i^t of each local gradient in $\mathcal{B}$ is calculated as follows:

$$\gamma_i^t = \frac{1}{2} \cdot (\alpha_i^t + \beta_i^t) \tag{12}$$

  which is used to weight the corresponding local gradient in the global model aggregation.
- Finally, the collaboration server sends γ_i^t and l_i^t for $i \in \mathcal{B}$ to the aggregation server.

The implementation of the above processes in the encryption environment is shown in Algorithm 3.

D. Global Model Updating

In this stage, the aggregation server globally aggregates the local gradients in $\mathcal{B}$. In order to improve the convergence speed and the accuracy of the global model, we design a novel global model updating method, which is divided into two stages: the aggregation of global gradient and the updating of global model parameters.

1) Aggregation of Global Gradient: In the aggregation stage of global gradient, the aggregation server firstly stores the benign local gradients and the reliability set $\mathcal{P}^t = \{\gamma_i^t\}$ with $i \in \mathcal{B}$ after the two-dimensional poisoning attack detection is executed for all the local gradients. Then each benign local gradient is weighted by the aggregation server according to its reliability. Finally, the global gradient $\boldsymbol{g}^t$ is obtained by aggregating these benign local gradients as follows:

$$\boldsymbol{g}^t = \sum_{i \in \mathcal{B}} \frac{\gamma_i^t}{\sum\limits_{j \in \mathcal{B}} \gamma_j^t} \cdot \boldsymbol{l}_i^t \tag{13}$$

2) Updating of Global Model Parameters: In this section, we show a detailed the updating process of global model parameters, whose purpose is to improve the convergence speed and the accuracy of the global model and reduce the decline of low-quality global gradients. The specific process is described as Algorithm 4. Before updating the global model, the aggregation server compares the reliability sum $\mathcal{A}^t = \sum_{i \in \mathcal{B}} \gamma_i^t$ of the benign local gradients in this t-round with the reliability sum $\mathcal{A}^{t-1}$ of the benign local gradients in the last round. If $\mathcal{A}^t \geq \mathcal{A}^{t-1}$, it indicates that the reliabilities of the global gradients in this t-round are relatively high, Otherwise, the reverse. Then, we normalize $\mathcal{A}^t - \mathcal{A}^{t-1}$ for more accurate updating as follows:

$$\sigma = \frac{\mathcal{A}^t - \mathcal{A}^{t-1}}{|\mathcal{A}^t - \mathcal{A}^{t-1}|} \tag{14}$$

if $\mathcal{A}^t - \mathcal{A}^{t-1} = 0$, we set $\sigma = 0$. Then, the update of global model parameters is divided into two cases:

- *case 1:* If $\sigma = -1$, we adjust the global gradient of this round by scaling the global gradient based on the cosine similarity δ^t between the current round of global gradient and the last round of global gradient as follows:

$$\boldsymbol{w}^t = \boldsymbol{w}^{t-1} - \eta \cdot \lambda^t \cdot \boldsymbol{g}^t \tag{15}$$

  where $\lambda^t = (\delta^t + 1)/2$, which is the normalized cosine similarity. Further, the detailed cosine similarity is calculated as follows:

$$\delta^t = \frac{\boldsymbol{g}^t \cdot \boldsymbol{g}^{t-1}}{||\boldsymbol{g}^t|| \cdot ||\boldsymbol{g}^{t-1}||} \tag{16}$$

- *case 2:* If $\sigma = 0$ or 1, there is no need to adjust the global gradient of this round, thus the following calculation is implemented:

$$\boldsymbol{w}^t = \boldsymbol{w}^{t-1} - \eta \cdot \boldsymbol{g}^t \tag{17}$$

Based on the above two cases, the global model can be updated according to the following equation:

$$\boldsymbol{w}^t = \boldsymbol{w}^{t-1} - \eta \cdot \varepsilon^t \cdot \boldsymbol{g}^t \tag{18}$$

where $\varepsilon^t = |\sigma - 0.5| \cdot (\lambda^t - 1) - 0.5 \cdot \lambda^t + 1.5$.

E. Global Model Distribution

In this section, we construct a secure global model distribution method, where the trained global model in each round can be

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

Algorithm 4: Global Model Updating.

Input : $[[\boldsymbol{w}^{t-1}]]_{pk_c}, \mathcal{B}, [[\boldsymbol{l}_i^t]]_{pk_c}, \boldsymbol{g}^{t-1} = [[\boldsymbol{g}^{t-1}]]_{pk_c}$ owned by AS, $sk_c, \mathcal{P}^t, \mathcal{P}^{t-1}$ owned by CS.
Output: Encrypted global model $[[\boldsymbol{w}^t]]_{pk_c}$.

Procedure:
  Aggregation server (AS):
for $i \in \mathcal{B}$ **do**
  $r_i \leftarrow$ Select a non-zero integer randomly
  $\boldsymbol{L}_i^t = [[r_i]]_{pk_c} \cdot [[\boldsymbol{l}_i^t]]_{pk_c}$
  Send $\boldsymbol{L}_i^t$ to CS
end for
  Collaboration server (CS):
for $i \in \mathcal{B}$ **do**
  $\mathbb{L}_i^t = Dec(sk_c, \boldsymbol{L}_i^t) \leftarrow$ Decrypt $\boldsymbol{L}_i^t$ with sk_c
  $\boldsymbol{h}_i^t = \frac{\gamma_i}{\sum_{j \in \mathcal{B}} \gamma_i} \cdot \mathbb{L}_i^t$
  $\boldsymbol{H}_i^t = [[\boldsymbol{h}_i^t]]_{pk_c} \leftarrow$ Encrypt $\boldsymbol{h}_i^t$ with pk_c
  Send $\boldsymbol{H}_i^t$ to AS
end for
  Aggregation server (AS):
for $i \in \mathcal{B}$ **do**
  $\boldsymbol{g}_i^t = (\boldsymbol{H}_i^t)^{\frac{-\gamma_i}{\sum_{j \in \mathcal{B}} \gamma_i}} \cdot [[r_i]]_{pk_c}^{-1}$
end for
Calculate $\boldsymbol{g}^t = \prod_{i \in \mathcal{B}} \boldsymbol{g}_i^t$
for $i \in \{t, t-1\}$ **do**
  $r_x \leftarrow$ Select a non-zero integer
  Calculate $\boldsymbol{G}^x = (\boldsymbol{g}^x)^{r_x}$
  Send $\boldsymbol{G}^x$ to CS
end for
  Collaboration server (CS):
for $x \in \{t, t-1\}$ **do**
  $\mathcal{A}^x = \sum_{i \in \mathcal{B}} \gamma_i^x$
end for
Calculate σ according to Eq. (14)
for $x \in \{t, t-1\}$ **do**
  Decrypt $\boldsymbol{G}^x$ with sk_c: $\mathbb{G}^x = Dec(sk_c, \boldsymbol{G}^x)$
end for
Calculate $\delta^t = \frac{\mathbb{G}^t \cdot \mathbb{G}^{t-1}}{||\mathbb{G}^t|| \cdot ||\mathbb{G}^{t-1}||}$
Calculate $\lambda^t = \frac{\delta^t + 1}{2}$
Calculate $\varepsilon^t = |\sigma - 0.5| \cdot (\lambda^t - 1) - 0.5 \cdot \lambda^t + 1.5$
Send ε^t to AS
  Aggregation server (AS):
/* ▷Update global model parameters */
$[[\boldsymbol{w}^t]]_{pk_c} = [[\boldsymbol{w}^{t-1}]]_{pk_c} \cdot (\boldsymbol{g}^t)_{pk_c}^{-\eta \cdot \varepsilon^t}$

Algorithm 5: Global Model Distribution.

Input : Encrypted global model $[[\boldsymbol{w}^t]]_{pk_c}$ owned by AS; private key sk_c owned by CS.
Output: $[[\boldsymbol{w}^t]]_{pk_i}, i \in \mathcal{N}$.

Procedure:
  Aggregation server (AS):
Randomly select a non-zero integer r
Calculate $\boldsymbol{W} = [[\boldsymbol{w}^t]]_{pk_c} \cdot [[r]]_{pk_c}$
Send $\boldsymbol{W}$ to CS
  Collaboration server (CS):
$\mathbb{W} = Dec(sk_c, \boldsymbol{W}) \leftarrow$ Decrypt $\boldsymbol{W}$ with sk_c
for $i \in \mathcal{N}$ **do**
  $\mathbb{W}_i^* = [[\mathbb{W}]]_{pk_i} \leftarrow$ Encrypt $\mathbb{W}$ with pk_i
  Send $\mathbb{W}_i^*$ to AS
end for
  Aggregation server (AS):
for $i \in \mathcal{N}$ **do**
  /* ▷Distribute the global model */
  $[[\boldsymbol{w}^t]]_{pk_i} = \mathbb{W}_i^* \cdot [[r]]_{pk_i}^{-1} \leftarrow$ Eliminate noise in $\mathbb{W}_i^*$
  Send $[[\boldsymbol{w}^t]]_{pk_i}$ to client u_i
end for

encrypted and distributed to each client. At the beginning of each training round, the aggregation server distributes the global model to clients, which is encrypted by the collaboration server's public key. However, the clients cannot decrypt it. Therefore, it's necessary to replace the collaboration server's public key with each client's public key to encrypt the global model. The detailed process is descried as follows (shown in Algorithm 5):

- *Step 1:* The aggregation server randomly selects a non-zero integer r as the data noise. Then the mask value of $[[\boldsymbol{w}^t]]_{pk_c}$ is calculated as follows:

$$\boldsymbol{W} \leftarrow [[\boldsymbol{w}^t]]_{pk_c} \cdot [[r]]_{pk_c}$$

  The aggregation server sends the $\boldsymbol{W}$ to the collaboration server.
- *Step 2:* The collaboration server receives and decrypts the $\boldsymbol{W}$ to obtain $\mathbb{W} \leftarrow Dec(sk_c, \boldsymbol{W})$. Although the collaboration server can decrypt the $\boldsymbol{W}$, it only can obtain the value $\mathbb{W}$, where $\mathbb{W}$ is seen as $\mathbb{W} = r + \boldsymbol{w}^t$, thus it cannot obtain the real global model $\boldsymbol{w}^t$. Then the collaboration server further encrypts the $\mathbb{W}$ by the public key of each client u_i to obtain $\mathbb{W}_i^* \leftarrow [[\mathbb{W}]]_{pk_i}$, and it sends the $\mathbb{W}_i^*$ to the aggregation server, respectively.
- *Step 3:* The aggregation server eliminates the data noise r in each $\mathbb{W}_i^*$ to obtain $[[\boldsymbol{w}^t]]_{pk_i} \leftarrow \mathbb{W}_i^* \cdot [[r]]_{pk_i}^{-1}$. Finally, the aggregation server sends $[[\boldsymbol{w}^t]]_{pk_i}$ to each client u_i, respectively.
- *Step 4:* Each client u_i decrypts the received $[[\boldsymbol{w}^t]]_{pk_i}$ with sk_i and then obtain the global model $\boldsymbol{w}^t$.

VI. Analysis of Proposed Scheme

In this section, the correctness and security of our federated learning scheme is analyzed.

A. Correctness

In this section, we show the correctness of our scheme. Our federated learning scheme can realize the detection of malicious local gradients based on homomorphic encryption, where all local gradients are in the encrypted and masked status on the aggregation server and the collaboration server. Also, our scheme can ensure that the computational result of the processed (encrypted and masked) local gradients is the same as the computational result of the original local gradients.

Theorem 2: Within the interactive computation processes between the aggregation server and the collaboration server, although the local gradients in the aggregation server are encrypted and the local gradients in the collaboration server are masked, these additional processes (addition and multiplication operations) have no influence on the correctness of the trained model results.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

Proof: According to the addition and multiplication operations required in the data (gradient) protection processes, we can prove Theorem 2 as follows:

- *Addition operations:* For $\mathbb{L}_i^t = Dec([[r_x]]_{pk_c} \cdot [[\boldsymbol{l}_i^t]]_{pk_c})$ with $i \in \{a, b\}$, we have $\mathbb{L}_i^t = r_x + \boldsymbol{l}_i^t, i \in \{a, b\}$ according to the homomorphism of encryption primitive. So, we can compute

$$\begin{aligned}\mathbb{L}_a^t - \mathbb{L}_b^t &= r_x + \boldsymbol{l}_a^t - (r_x + \boldsymbol{l}_b^t) \\ &= \boldsymbol{l}_a^t - \boldsymbol{l}_b^t\end{aligned}$$

- *Multiplication operations:* For $\mathbb{L}_i^t = Dec([[\boldsymbol{l}_i^t]]_{pk_c}^{r_g^t})$, with $t \in \{a, b\}$, where r_g^t is a random integer with $t \in \{a, b\}$. Then, according to the homomorphism of encryption primitive, we have $\mathbb{L}_i^t = r_g^t \cdot \boldsymbol{l}_i^t$. Further, we can calculate

$$\mathbb{L}_i^a \cdot \mathbb{L}_i^b = r_g^a \cdot r_g^b \cdot (\boldsymbol{l}_i^a \cdot \boldsymbol{l}_i^b)$$

$$\begin{aligned}||\mathbb{L}_i^a|| \cdot ||\mathbb{L}_i^b|| &= (r_g^a \cdot ||\boldsymbol{l}_i^a||) \cdot (r_g^b \cdot ||\boldsymbol{l}_i^b||) \\ &= r_g^a \cdot r_g^b \cdot (||\boldsymbol{l}_i^a|| \cdot ||\boldsymbol{l}_i^b||)\end{aligned}$$

Therefore, we can get $\frac{\mathbb{L}_i^a \cdot \mathbb{L}_i^b}{||\mathbb{L}_i^a|| \cdot ||\mathbb{L}_i^b||} = \frac{\boldsymbol{l}_i^a \cdot \boldsymbol{l}_i^b}{||\boldsymbol{l}_i^a|| \cdot ||\boldsymbol{l}_i^b||}$.

In summary, the addition and multiplication operations required in the data protection processes have no influence on the correctness of the trained model results. ■

B. Security

In this section, the security of our scheme is analyzed. It is proved that our scheme can provide the privacy-preserving security for the local gradients calculated by the clients, where the interactive computations are processed by the aggregation server and the collaboration server.

Theorem 3: Our scheme is able to protect the privacy of the local gradients calculated by the benign clients, where the poisoners, the aggregation server and the collaboration server do not reveal any information about the benign clients' privacy data.

Proof: Similar to the works [9], [42], we prove this theorem by using a standard mixed argument. Given a security parameter k, a client set U and two non-colluding servers (aggregation server and collaboration server) denoted as $\mathrm{T} = \{\mathrm{AS}, \mathrm{CS}\}$, we set $\mathrm{Real}_{\mathrm{T}}^{U,k}$ as a random variable to represent the joint view of T when the above protocol is actually executed. Then, we prove that for all ks and T, there is a probabilistic polynomial time (PPT) simulator SIM, whose output (i.e., $\mathrm{SIM}_{\mathrm{T}}^{U,k}$) is no different from real $\mathrm{Real}_{\mathrm{T}}^{U,k}$.

Hyb1: A random variable is initialized, whose distribution is no different from $\mathrm{Real}_{\mathrm{T}}^{U,k}$, the joint view of the parties T in the practical operation of the protocol.

Hyb2: The behavior of each client $u_i \in U$ is changed, where client u_i encrypts a randomly selected vector $\boldsymbol{\rho}_i^t$ with pk_c under the Paillier cryptosystem, instead of executing $[[\boldsymbol{l}_i^t]]_{pk_c}$. The non-colluding setup for AS and CS and the Paillier cryptosystem make that this hybrid is undifferentiated from the last one.

Hyb3: The aggregation server is simulated to perturb $\boldsymbol{\rho}_i^t$ with noise ζ_i^t, rather than computing $[[r_x]]_{pk_c} \cdot [[\boldsymbol{l}_i^t]]_{pk_c}$. The non-colluding setup for AS and CS and the Paillier cryptosystem make that this hybrid is undifferentiated from the last one.

Hyb4: The input of the Algorithm 2 operated by the aggregation server and the collaboration server is changed to $[[\boldsymbol{\rho}_i^t]]_{pk_c}$ rather than $[[\boldsymbol{l}_i^t]]_{pk_c}$. The security of the Algorithm 2 guarantees the aggregation server and the collaboration server know nothing about $\boldsymbol{\rho}_i^t$. Thus, this hybrid is undifferentiated from the last one.

Hyb5: The aggregation server is simulated to compute $[[\boldsymbol{\rho}_i^t]]_{pk_c}^{\zeta_i^t}$, rather than computing $[[\boldsymbol{l}_i^t]]_{pk_c}^{r_g}$. The non-colluding setup for AS and CS and the Paillier cryptosystem make that this hybrid is undifferentiated from the last one.

Hyb6: The input of the Algorithm 4 operated by the aggregation server and the collaboration server is changed to $[[\boldsymbol{\tau}^{t-1}]]_{pk_c}$, $[[\boldsymbol{\rho}_i^t]]_{pk_c}$ and $[[\boldsymbol{\mu}^{t-1}]]_{pk_c}$, rather than $[[\boldsymbol{w}^{t-1}]]_{pk_c}$, $[[\boldsymbol{l}_i^t]]_{pk_c}$ and $[[\boldsymbol{g}^{t-1}]]_{pk_c}$. The non-colluding setup for AS and CS and the Paillier cryptosystem make that this hybrid is undifferentiated from the last one.

Hyb7: The aggregation server is simulated to eliminate the noise ζ_i^t in $[[\boldsymbol{\tau}_i^t]]_{pk_c}$, rather than computing $(\boldsymbol{H}_i^t)^{\frac{-\gamma_i}{\sum_{j\in\mathcal{B}}\gamma_i}} \cdot [[r_i]]_{pk_c}^{-1}$. The non-colluding setup for AS and CS and the Paillier cryptosystem make that this hybrid is undifferentiated from the last one.

Hyb8: The aggregation server is simulated to compute $\prod_{i\in\mathcal{B}} \boldsymbol{\tau}_i^t$ rather than $\prod_{i\in\mathcal{B}} \boldsymbol{g}_i^t$. The non-colluding setup for AS and CS and the Paillier cryptosystem make that this hybrid is undifferentiated from the last one.

Hyb9: Similar to **Hyb5**, The aggregation server is simulated to compute $[[\boldsymbol{\tau}_i^t]]_{pk_c}^{\zeta_i^t}$ rather than $(\boldsymbol{g}^x)^{r_x}$. The non-colluding setup for AS and CS and the Paillier cryptosystem make that this hybrid is undifferentiated from the last one.

Hyb10: The input of the Algorithm 5 operated by the aggregation server and the collaboration server is changed to $[[\boldsymbol{\tau}^t]]_{pk_c}$ rather than $[[\boldsymbol{w}^t]]_{pk_c}$. The non-colluding setup for clients, AS and CS and the Paillier cryptosystem make that this hybrid is undifferentiated from the last one.

Hyb11: Similar to **Hyb3**, we simulate the aggregation server to perturb $\boldsymbol{\tau}_i^t$ with noise ζ_i^t, rather than computing $[[\boldsymbol{w}^t]]_{pk_c} \cdot [[r]]_{pk_c}$. The non-colluding setup for clients, AS and CS and the Paillier cryptosystem make that this hybrid is undifferentiated from the last one.

As described above, we can prove that there is a probabilistic polynomial time (PPT) simulator SIM, whose output (i.e., $\mathrm{SIM}_{\mathrm{T}}^{U,k}$) is no different from real $\mathrm{Real}_{\mathrm{T}}^{U,k}$, where the aggregation server and the collaboration server know nothing about the benign clients' privacy data.

VII. Performance Analysis

In this section, some experiments is conducted on both target and untarget attacks to evaluate the performance of our proposed scheme by the real-world datasets. All experiments are run on Lenovo servers, configured with AMD Ryzen 7 5800H with Radeon Graphics 3.20 GHz CPU and 16 GB of RAM.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

TABLE II
MODEL STRUCTURES OF LENET-5

Layer	Input shape	Output shape
Convolutional+Pooling	[1,1,32,32]	[1,6,14,14]
Convolutional+Pooling	[1,6,14,14]	[1,16,5,5]
Full connection	[1,400]	[1,120]
Full connection	[1,120]	[1,84]
Full connection	[1,84]	[1,10]

A. Experimental Setting

1) Datasets: We use two datasets to verify the performance of proposed scheme:

- *MNIST:* The MNIST dataset has a total of 70000 sheets 28×28 sized handwritten digital images, all of which are grayscale images, with a total of 10 categories consisting of ten natural numbers ranging from 0 to 9. Furthermore, MNIST includes 60000 images for training and 10000 images for testing.
- *Fashion-MNIST:* Fashion-MNIST has the same attributes as MNIST except for the type of samples. The main difference between them is that Fashion-MNIST is a dataset that includes clothing categories such as T-shirts and pants et al. while MNIST is a dataset that includes handwritten digits.

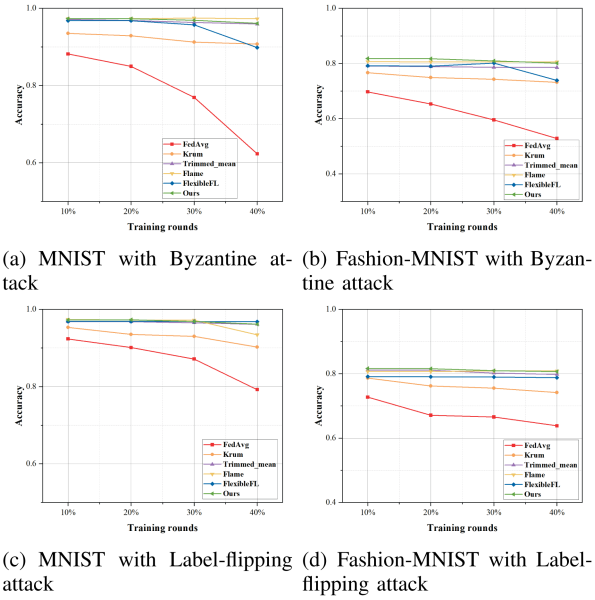
2) Attack Setting: In this section, we set target and untarget attacks as follows. For untarget attacks, we mainly verify the defense performance of our scheme against Byzantine attacks. We randomly generate the local gradients for each malicious client in the Byzantine attack based on a standard normal distribution.[5] For target attacks, we verify the defense performance of our scheme against Label-flipping attacks. We process the datasets of the poisoners in the Label-flipping attack as follows: we replace the samples' labels $\widehat{y}$ to $(\widehat{y}+Q-1)\%Q$, where $\widehat{y} \in \{0, 1, \ldots, Q-1\}$ and Q is the quantity of types of label.

3) Federated Learning Setting: In our experiment, we set the training environment of federated learning, where the proportion of poisoners is set from 10% to 40%. We use the model LeNet-5 [43] as the model for training, whose structure is introduced in Table II in details. The parameters batch_set size is set to 64 and momentum is set to 0.9. Moreover, we use the FedAvg scheme with no defend as the baseline, and compare our scheme with the Trimmed_mean [13] and Krum [14] schemes in IID setting. Correspondingly, we conduct a comparison with the FlexibleFL [15], SVFLC [44], and FLAME [45] under the Non-IID scenario.

B. Experimental Result

In this section, we evaluate the performance of our proposed scheme through our set experiments. Model performance is influenced by multiple factors, though our experiment specifically focuses on the defense mechanism rather than client data quality.

[5] the normal distribution whose mean is 0 and variance is 1.



(a) MNIST with Byzantine attack (b) Fashion-MNIST with Byzantine attack

(c) MNIST with Label-flipping attack (d) Fashion-MNIST with Label-flipping attack

Fig. 7. Model accuracy comparison with different attacks, datasets and proportions of poisoners.

We evaluate the proposed scheme on two datasets, examining how the proportion of malicious clients, the number of iteration rounds, the total client count, and the choice of aggregation method impact the trained model. A functional comparison with existing schemes is provided in Table III. In this work, fairness is defined as the defense scheme's ability to balance effective attack mitigation with the maintenance of high performance for all honest participants.

1) Accuracy: In this section, the influence of different factors on the accuracy of our scheme is tested.

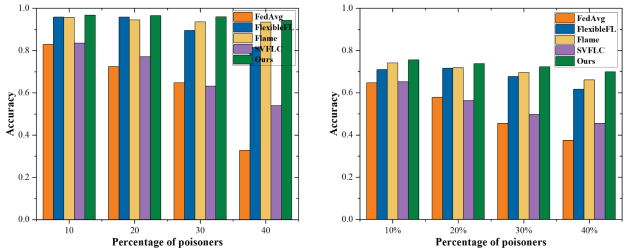
a) Proportion of Poisoners: In this section, we evaluate the impact of the proportion of poisoners on the trained model. As the proportion increases, the effect on model training becomes more significant. Depending on the model's capacity, issues such as underfitting or overfitting may arise. As shown in Fig. 7, in the IID setting, we compare the model accuracies generated by different schemes under different attacks, datasets, and proportions of poisoners. When the poisoning rate is below 30%, our scheme maintains the best detection performance compared to the other schemes. When the rate exceeds 30%, while the defensive performance of our scheme declines slightly, it is surpassed only by FLAME scheme and continues to outperform the remaining schemes.

As shown in Fig. 8, in the Non-IID setting, our proposed scheme achieves higher global model accuracy compared to several other schemes under both Byzantine attacks and label-flipping attacks. Meanwhile, as the proportion of poisoning increases, the model accuracy of different schemes decreases. Only FLAME scheme and our scheme still maintain high accuracy. When the poisoning proportion exceeds 30%, the accuracy gap between our scheme and FLAME scheme narrows. This is because FLAME scheme aggregates only the largest cluster, which theoretically allows it to tolerate nearly 50% malicious clients. This improves robustness, but it adversely affects the

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

TABLE III
FUNCTIONAL COMPARISON WITH EXISTING SCHEMES

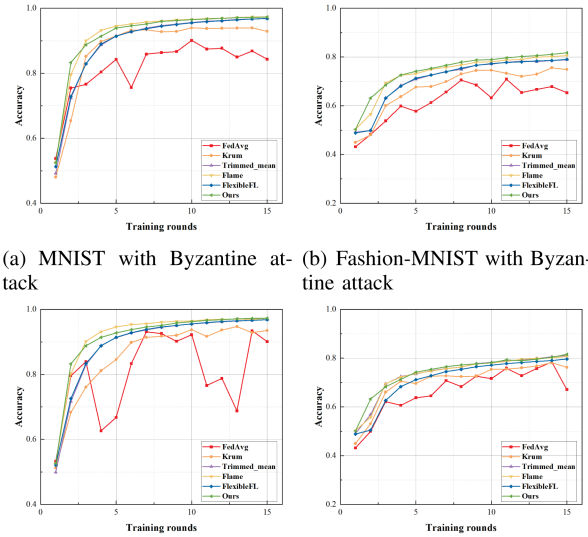
Function	Privacy preserving	Collusion resistance	Handling Non-IID	Fairness	Aggregation result Privacy
[46],[47]	✗	✓	✓	✓	✗
[48]	✗	✓	✓	✗	✗
[49]	✗	✓	✓	✓	✗
[50]	✗	✓	✗	✗	✗
[51],[52]	✓	✓	✗	✗	✗
[53]	✓	✓	✗	✗	✓
[54],[55]	✓	✗	✓	✓	✓
Ours	✓	✓	✓	✓	✓



(a) MNIST with Byzantine attack (b) Fashion-MNIST with Byzantine attack

(c) MNIST with Label-flipping attack (d) Fashion-MNIST with Label-flipping attack

Fig. 8. Model accuracy comparison with different attacks, datasets and aggregation methods under Non-IID setting.



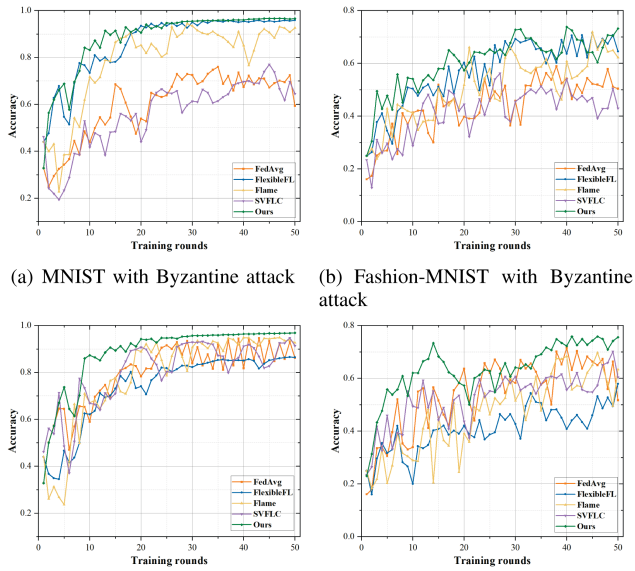
(a) MNIST with Byzantine attack (b) Fashion-MNIST with Byzantine attack

(c) MNIST with Label-flipping attack (d) Fashion-MNIST with Label-flipping attack

Fig. 9. Model accuracy comparison under IID setting.

model convergence. Our scheme first needs to detect a portion of malicious local gradients, and then weights the remaining local gradients for global aggregation. This way is slightly weaker when the proportion of poisoners is high.

b) Iteration Rounds: In this section, the influence of iteration rounds on the trained model is evaluated. As the iteration Rounds increases, the prediction accuracy of the trained model also increases. This is because more positive data features can be continuously extracted by the model during training process. We use a malicious client ratio of 20% as an example to observe how does the iteration rounds affect the model accuracy. Fig. 9 shows the influence of iteration rounds on model accuracy under IID setting. As shown in Fig. 9, as the iteration rounds increases, the accuracy of the models generated by all evaluated defense schemes gradually increases. Furthermore, as we can see, the overall accuracy of our scheme shows no significant difference from that of FLAME scheme, and it outperforms the other schemes. As shown in Fig. 10, the model accuracy of all compared schemes increases with the number of training rounds. Among them, our scheme consistently achieves better performance and greater stability throughout the training process.

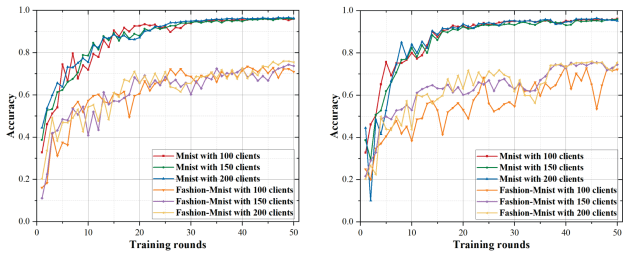


(a) MNIST with Byzantine attack (b) Fashion-MNIST with Byzantine attack

(c) MNIST with Label-flipping attack (d) Fashion-MNIST with Label-flipping attack

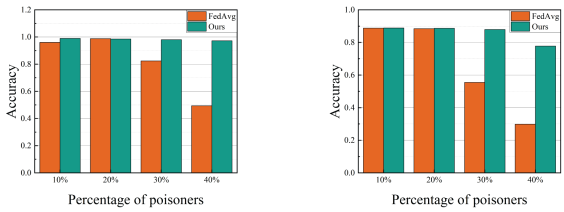
Fig. 10. Model accuracy comparison under Non-IID setting.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

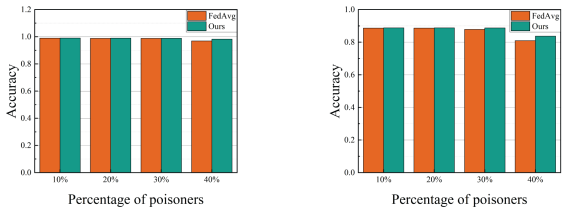


(a) Different datasets with Byzantine attack (b) Different datasets with Label-flipping attack

Fig. 11. Comparison of model accuracy with different numbers of clients under Non-IID setting.



(a) MNIST with Byzantine attack (b) Fashion-MNIST with Byzantine attack



(c) MNIST with Label-flipping attack (d) Fashion-MNIST with Label-flipping attack

Fig. 12. Model accuracy comparison with different attacks, datasets and aggregation methods.

c) Client Counts: In this section, the influence of client counts on the trained model is evaluated. The experimental results about the influence of client counts on model accuracy with 100, 150 and 200 clients are shown in Fig. 11. We use a malicious client ratio of 30% as an example to observe how does the number of clients affect the model accuracy. And we evaluated the model's performance on the MNIST and Fashion-MNIST datasets, considering both Byzantine and label-flipping attacks. As can be seen from the figure, an increase in the number of clients has almost no impact on the model's accuracy.

d) Different Aggregation Methods: In this section, the influence of aggregation methods on the trained model is tested. To be specific, we test the aggregation effectiveness under our scheme and the federated learning scheme (FedAvg), where our two-dimensional malicious local gradient detection method is adaptively added to the FedAvg scheme. Fig. 12 shows the comparison of model prediction accuracies generated by our scheme and the FedAvg scheme. From the Fig. 12, it can be seen that whether it is Label-flipping attack or Byzantine attack, the global model's prediction accuracy under our aggregation scheme and the FedAvg scheme is different based on our malicious local gradient detection method. The trained model performance of our aggregation scheme is better than that of the FedAvg scheme. Also, what can be learned that as the proportion of poisoners increases, the difference of prediction accuracy between the global models obtained by two schemes becomes larger. Especially, if the proportion of poisoners exceeds 30%, the difference significantly increases. That is because we set two-dimensional malicious local gradient detection in our full detection procedure. When the benign probabilities for related local gradients are smaller than the corresponding first-quartile of all local gradients, these local gradients are considered to be malicious, which results in our scheme can confirm up to 25% of the local gradients as malicious every time. Also, when the proportion of poisoners exceeds 25%, there may be some malicious local gradients that are not detected, which causes malicious local gradients that evade detection to continue to damage the global model. Compared with the FedAvg scheme, our scheme adds a weighting mechanism for malicious client detection, where the weights of malicious local gradients that are not detected are set to lower values according to their reliability. This can further guarantee that the performance of the global model is not compromised by malicious local gradients. So, the aggregation procedure under our scheme is effective.

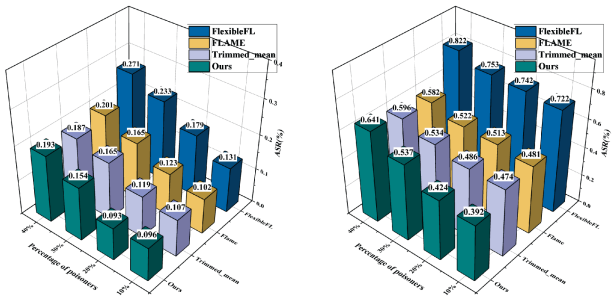
2) Training Loss: In this section, we detail how the average training loss of all clients is affected by the proportion of poisoners, where we compare our scheme with the traditional federated learning scheme without any defense mechanisms (baseline). As show in Table IV, the average training loss exhibits the same trend in both IID and Non-IID settings. From the Table IV, it can be seen that the training loss can be effectively reduced by our scheme for Byzantine attacks in various datasets. However, for Label-flipping attacks, Table IV shows that the gap of loss between our scheme and the baseline is small. As the proportion of poisoners increases, the loss also increases. We analyze the reason for the above situation is that 1) we do not need to process each local dataset for Byzantine attacks, namely the data samples in each local dataset are normal; 2) we implement the poisoning attack by directly randomly generating local gradient of each malicious client and uploading the gradients to the aggregation server; 3) for Label-flipping attacks, we processed the dataset of poisoners by changing labels, which causes the changes in the original dataset of poisoners to generate a significant loss for poisoners (thereby affecting the overall loss).

3) Attack Success Rate: In this section, we evaluate the defense performance of our proposed scheme against label-flipping attacks under both IID and Non-IID settings. The poisoning rate ranges from 10% to 40%, and the attack success rate (ASR) is used as the evaluation metric. As shown in Fig. 13, the ASR generally increases with the poisoning rate across all defense schemes. At poisoning rates below 30%, our scheme maintains the lowest ASR. However, when the rate reaches 40%, its ASR is slightly higher than those of the FLAME and trimmed_mean schemes. Furthermore, Fig. 14 illustrates that in the Non-IID setting, all four defense methods exhibit an increasing ASR trend as the poisoning rate rises. However, our scheme achieves the lowest ASR across both datasets, demonstrating superior robustness. These results confirm that our scheme provides effective defense against label-flipping attacks.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

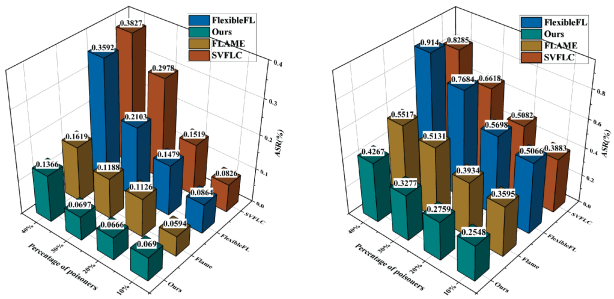
TABLE IV
AVERAGE TRAINING LOSS UNDER DIFFERENT SITUATIONS

Datasets	Attacks	Methods	Proportion of poisoners (IID)				Proportion of poisoners (Non-IID)			
			10%	20%	30%	40%	10%	20%	30%	40%
MNIST	Byzantine	Baseline	0.1318	0.7526	1.9867	8.597	0.1409	0.2462	0.3836	0.8076
		Ours	0.0104	0.0078	0.0475	0.0601	0.0121	0.0114	0.0258	0.0571
	Label-flipping	Baseline	0.0176	0.0194	0.0286	0.0256	0.0177	0.0146	0.0159	0.0176
		Ours	0.0157	0.0123	0.0147	0.0238	0.0112	0.0165	0.0105	0.0203
Fashion-MNIST	Byzantine	Baseline	0.3856	0.8983	1.6547	4.5064	0.1570	0.2869	0.4395	0.5157
		Ours	0.2475	0.2457	0.2416	0.3236	0.0825	0.0768	0.0978	0.1067
	Label-flipping	Baseline	0.2325	0.2511	0.2631	0.2708	0.0691	0.0799	0.0732	0.0784
		Ours	0.1824	0.1999	0.2353	0.2605	0.0795	0.1035	0.0710	0.0848



(a) MNIST with Label-flipping attack (b) Fashion-MNIST with Label-flipping attack

Fig. 13. ASR comparison with different datasets and poisoning rates under IID setting.

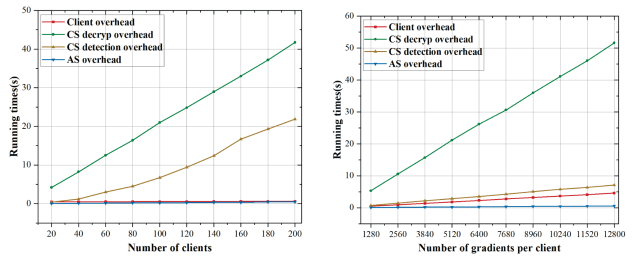


(a) MNIST with Label-flipping attack (b) Fashion-MNIST with Label-flipping attack

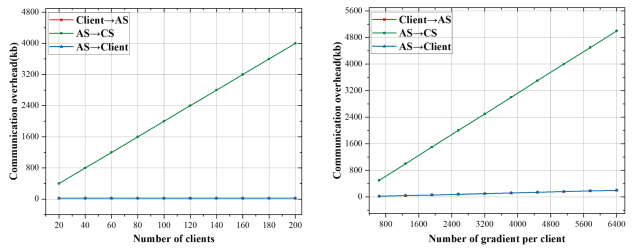
Fig. 14. ASR comparison with different datasets and poisoning rates under Non-IID setting.

4) Overhead Analysis: In this section, we analyze the impact of different numbers of clients and gradients on the computation and communication overhead of our scheme.

a) Computation overhead: In this section, we evaluate how the computation overhead varies with the number of clients and gradients. Fig. 15(a) and Fig. 15(b) illustrate that the computational overhead grows linearly as the number of clients and gradients increases, which is dominated by the decryption overhead at the collaboration server because it must decrypt all perturbed and encrypted gradients. Consequently, our scheme manages to defend against poisoning attacks and preserve client privacy with a reasonable computation overhead.



(a) Computation overhead of client with the number of clients (b) Computation overhead of client with the number of gradients



(c) Communication overhead of client with the number of clients (d) Communication overhead of client with the number of gradients

Fig. 15. Computation and communication overhead comparison.

b) Communication overhead: In this section, we evaluate the impact of an increasing number of clients and gradients on the communication overhead. As shown in Fig. 15(c) and Fig. 15(d), there is a linear increase in communication overhead corresponding to the growth in the number of gradients and clients. The aggregation server bears the largest portion of this overhead because it must transmit the perturbed gradients from all clients to the collaboration server to detect malicious gradients, and simultaneously broadcast the encrypted global gradient to all clients. Consequently, this level of communication overhead is deemed acceptable for preserving client privacy.

VIII. CONCLUSION

Although federated learning provides a convenient model training framework for distributed machine learning, it also brings some security concerns: the security of model training procedure and the privacy of clients' data. So, in this paper we propose the two-dimensional privacy-preserving federated

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

learning scheme against poisoning attacks, which can protect the clients' privacy data by homomorphic encryption and detect malicious gradients by two-dimensional detection method in the model training. We first propose a two-dimensional detection method for data poisoning attacks, which can detect local gradients accurately according to the distribution of local gradients and the update direction of each local gradient. Also, we construct an improved global model aggregation method to weight the corresponding gradient based on the reliability of each local gradient and adjust the update amplitude of the global gradient in each round, which can improve the convergence speed of the global model and ensure the prediction accuracy of the global model. Related experimental results show that our scheme is effective against two types of poisoning attacks. In our future work, we will investigate how to combine our defense scheme with more efficient cryptographic schemes, such as lattice-based cryptography, to enhance its scalability for large-scale federated learning scenarios.

REFERENCES

[1] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," 2016, *arXiv:1610.05492*.

[2] J. Konečný, H. B. McMahan, D. Ramage, and P. Richtárik, "Federated optimization: Distributed machine learning for on-device intelligence," 2016, *arXiv:1610.02527*.

[3] B. Hitaj, G. Ateniese, and F. Perez-Cruz, "Deep models under the gan: Information leakage from collaborative deep learning.," in *Proc. ACM SIGSAC conf. comput. commun. secur.*, 2017, pp. 603–618.

[4] J. Geiping, H. Bauermeister, H. Dröge and M. Moeller, "Inverting gradients- how easy is it to break privacy in federated learning?," *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 16937–16947.

[5] X. Liu, H. Li, G. Xu, S. Liu, Z. Liu, and R. Lu, "PADL: Privacy-aware and asynchronous deep learning for IoT applications," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 6955–6969, Aug., 2020.

[6] R. Shokri and V. Shmatikov, "Privacy-preserving deep learning," in *Proc. 53rd Annu. Allerton Conf. Commun., Control, Comput. (Allerton)*, 2015, pp. 1310–1321.

[7] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. 2017 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1175–1191.

[8] F. Zheng, C. Chen, X. Zheng, and M. Zhu, "Towards secure and practical machine learning via secret sharing and random permutation," *Knowl. Based Syst.*, vol. 245, 2022, Art. no. 108609.

[9] G. Xu, H. Li, Y. Zhang, S. Xu, J. Ning, and R. H. Deng, "Privacy-preserving federated deep learning with irregular users," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1364–1381, Mar./Apr., 2022.

[10] A. Shafahi et al., "Poison frogs! targeted clean-label poisoning attacks on neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 6106–6116.

[11] J. Zhang, B. Chen, X. Cheng, H. T. T. Binh, and S. Yu, "PoisonGAN: Generative poisoning attacks against federated learning in edge computing systems," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3310–3322, Mar., 2021.

[12] X. Cao and N. Z. Gong, "MPAF: Model poisoning attacks to federated learning based on fake clients," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, 2022, pp. 3395–3403.

[13] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5650–5659.

[14] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Neural Inf. Process. Syst.*, 2017, pp. 119–129.

[15] Y. Zhao, Y. Cao, J. Zhang, H. Huang, and Y. Liu, "FlexibleFL: Mitigating poisoning attacks with contributions in cloud-edge federated learning systems," *Inf. Sci.*, vol. 664, 2024, Art. no. 120350.

[16] K. Wei et al., "Federated learning with differential privacy: Algorithms and performance analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 3454–3469, 2020.

[17] L. T. Phong, Y. Aono, T. Hayashi, L. Wang, and S. Moriai, "Privacy-preserving deep learning via additively homomorphic encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 5, pp. 1333–1345, May, 2018.

[18] Y. Miao, Z. Liu, H. Li, K. -K. R. Choo, and R. H. Deng, "Privacy-preserving byzantine-robust federated learning via blockchain systems," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 2848–2861, 2022.

[19] D. Chen, H. Qu, and G. Xu, "AegisFL: Efficient and flexible privacy-preserving byzantine-robust cross-silo federated learning," in *Proc. Forty-First Int. Conf. Mach. Learn.*, 2024, pp. 7207–7219.

[20] X. Liu, H. Li, G. Xu, Z. Chen, X. Huang, and R. Lu, "Privacy-enhanced federated learning against poisoning adversaries," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4574–4588, 2021.

[21] C. Chen, S. Babakniya, M. Paolieri, and L. Golubchik, "Defending against poisoning backdoor attacks on federated meta-learning," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 5, 2022, Art. no. 76.

[22] Y. Wang, D. Zhai, Y. He, and Y. Xia, "An adaptive robust defending algorithm against backdoor attacks in federated learning," *Future Gener. Comput. Syst.*, vol. 143, pp. 118–131, 2023.

[23] B. Hou et al., "Mitigating the backdoor attack by federated filters for industrial IoT applications," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3562–3571, May, 2022.

[24] C. Zhu, J. Zhang, X. Sun, B. Chen, and W. Meng, "ADFL: Defending backdoor attacks in federated learning via adversarial distillation," *Comput. Secur.*, vol. 132, 2023, Art. no. 103366.

[25] N. M. Jebreel and J. Domingo-Ferrer, "Fl-defender: Combating targeted attacks in federated learning," *Knowl. Based Syst.*, vol. 260, 2023, Art. no. 110178.

[26] S. Li, E. Ngai, and T. Voigt, "Byzantine-robust aggregation in federated learning empowered industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1165–1175, Feb., 2023.

[27] X. Cao, Z. Zhang, J. Jia, and N. Z. Gong, "FLCert: Provably secure federated learning against poisoning attacks," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 3691–3705, 2022.

[28] P. Kairouz and H. B. McMahan, "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1-2, pp. 1–210, 2021.

[29] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *EUROCRYPT*, Lecture Notes in Computer Science, 1999, vol. 1592, pp. 223–238.

[30] Y. Miao et al., "RFed: Robustness-enhanced privacy-preserving federated learning against poisoning attack," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 5814–5827, 2024.

[31] X. Li, X. Yang, Z. Zhou, and R. Lu, "Efficiently achieving privacy preservation and poisoning attack resistance in federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 4358–4373, 2024.

[32] H. Zeng et al., "ESFL: Accelerating poisonous model detection in privacy-preserving federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 4, pp. 3780–3794, Jul./Aug., 2025.

[33] R. Xu, B. Li, C. Li, J. B. D. Joshi, S. Ma, and J. Li, "TAPFed: Threshold secure aggregation for privacy-preserving federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 5, pp. 4309–4323, Sep./Oct., 2024.

[34] Z. Ma, J. Ma, Y. Miao, Y. Li, and R. H. Deng, "ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1639–1654, 2022.

[35] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Int. Conf. Adv. Neural Inf. Process. Syst.*, 2017, pp. 119–129.

[36] E. M. E. Mhamdi, R. Guerraoui, and S. Rouault, "The Hidden Vulnerability of Distributed Learning in Byzantium," in *Proc. Int. Conf. Mach. Learn.*, in Proceedings of Machine Learning Research, 2018, vol. 80, pp. 3518–3527.

[37] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. Int. Conf. Mach. Learn.*, in Proceedings of Machine Learning Research, 2018, vol. 80, pp. 5636–5645.

[38] M. Fang et al., "Local model poisoning attacks to byzantine-robust federated learning," in *USENIX Secur. Symposium. USENIX Assoc.*, 2020, pp. 1605–1622.

[39] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," in *Proc. Int. Conf. Mach. Learn.*, in Proceedings of Machine Learning Research, 2021, vol. 139, pp. 6357–6368.

[40] C. Xie, O. Koyejo, and I. Gupta, "Fall of empires: Breaking byzantine-tolerant SGD by inner product manipulation," in *UAI*, in Proceedings of Machine Learning Research, AUAI Press, 2019, vol. 115, pp. 261–270.

[41] B. W. Silverman, "Density estimation for statistics and data analysis," *Monographs Statist. Appl. Probability*, 1986.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.

[42] C. Gentry, J. Groth, Y. Ishai, C. Peikert, A. Sahai, and A. Smith, "Using fully homomorphic hybrid encryption to minimize non-interative zero-knowledge proofs," *J. Cryptol.*, vol. 28, no. 4, pp. 820–843, 2015.
[43] Y. Lecun and L. Bottou, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov., 1998.
[44] N. Li, M. Zhou, H. Yu, Y. Chen, and Z. Yang, "SVFLC: Secure and verifiable federated learning with chain aggregation," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 13125–13136, Apr., 2024.
[45] T. D. Nguyen et al., "FLAME: Taming backdoors in federated learning," in *Proc. 31st USENIX Secur. Symp. (USENIX Secur. 22)*, 2022, pp. 1415–1432.
[46] Y. Mao, X. Yuan, X. Zhao, and S. Zhong, "Romoa: Robust model aggregation for the resistance of federated learning to model poisoning attacks," in *Proc. Comput. Secur.–ESORICS 2021*, 2021, pp. 476–496.
[47] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "Mitigating sybils in federated learning poisoning," 2018, *arXiv:1808.04866*.
[48] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," 2020, *arXiv:2012.13995*.
[49] S. Awan, B. Luo, and F. Li, "CONTRA: Defending against poisoning attacks in federated learning," in *Proc. Comput. Secur.–ESORICS 2021*, 2021, pp. 455–475.
[50] P. Blanchard, R. Guerraoui, J. Stainer, and E. M. E. Mhamdi, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 119–129.
[51] M. Srewa, M. F. Winfree, M. I. Ibrahem, M. Nabil, R. Lu, and A. Alsharif, "Poisoning attack mitigation for privacy-preserving federated learning-based energy theft detection," in *Proc. IEEE Int. Conf. Commun.*, 2024, pp. 3962–3968.
[52] M. I. Ibrahem, M. M. Fouda, and Z. M. Fadlullah, "FedSafe-no KDC needed: Decentralized federated learning with enhanced security and efficiency," in *Proc. IEEE 21st Consum. Commun. Netw. Conf.*, 2024, pp. 969–975.
[53] D. Namakshenas, A. Yazdinejad, A. Dehghantanha, R. M. Parizi, and G. Srivastava, "IP2FL: Interpretation-based privacy-preserving federated learning for industrial cyber-physical systems," *IEEE Trans. Ind. Cyber-Phys. Syst.*, 2024, vol. 2, pp. 321–330.
[54] A. Yazdinejad, A. Dehghantanha, and G. Srivastava, "AP2FL: Auditable privacy-preserving federated learning framework for electronics in healthcare," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 2527–2535, Feb., 2024.
[55] A. Yazdinejad, A. Dehghantanha, H. Karimipour, G. Srivastava, and R. M. Parizi, "A robust privacy-preserving federated learning model against model poisoning attacks," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 6693–6708, 2024.



Ke Gu (Member, IEEE) received the PhD degree from the School of Information Science and Engineering, Central South University, Changsha, China, in 2012. In 2013, he joined the School of Computer Science and Technology, Changsha University of Science and Technology, where he is an associate professor. He has authored or coauthored more than 100 research papers in journals and conferences. His research interests include artificial intelligence security, IoVs, VANETs, social networks, and network and information security.



Wenwu Zhao received the BS degree from the East China University of Technology, Nanchang, China, in 2023. He is currently working toward the MS degree in computer technology with the School of Computer Science, Changsha University of Science & Technology, Changsha, China. His research interests include artificial intelligence security and privacy preservation.



Jingjing Tan (Member, IEEE) received the BE and MS degrees from the School of Computer Science and Technology, Changsha University of Science and Technology, Changsha, China, in 2017 and 2020, respectively, and the PhD degree from the School of Computer Science and Engineering, Central South University, Changsha, in 2024. He is currently an assistant professor with the School of Computer Science and Technology, Changsha University of Science and Technology. His research interests include wireless mobile communications, artificial intelligence security, and network and information security.



Zheng Yang received the MS degree from the School of Computer Science and Technology, Changsha University of Science and Technology, Changsha, China, in 2024. His research interests include privacy preservation and federated learning.



Xiong Li (Senior Member, IEEE) received the PhD degree in computer science and technology from the Beijing University of Posts and Telecommunications, Beijing, China, in 2012. He is currently a full professor with the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China. He has authored or coauthored more than 150 referred papers. His current research interests include artificial intelligence security, cryptography, and information security.



WeiJia Jia (Fellow, IEEE) received the PhD degree in computer science from the Polytechnic Faculty of Mons, Mons, Belgium, in 1993. He is currently a full professor with the Institute of Artificial Intelligence and Future Networks, Beijing Normal University. He has authored or coauthored more than 500 research papers in journals or conferences. His research interests include next-generation wireless communication, artificial intelligence, distributed systems, and multicast and anycast routing protocols.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 05:33:22 UTC from IEEE Xplore. Restrictions apply.